Cloud Computing Project Based Learning (PBL) Report on

SECURE CLOUD FILE STORAGE USING ENCRYPTION

Submitted in partial fulfilment of

Requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING - CYBER SECURITY

Submitted by

D.Pavani Reddy	**23R11A62A4**
D.Alekhya Ashwini	**23R11A62A6**
J.Lasya Priya	**23R11A62B6**

Under the esteemed guidance

Of

V. Soujenya

Assistant Professor

Department of CSE - Cyber Security



Geethanjali College of Engineering and Technology

Department of Computer Science and Engineering - Cyber Security

(UGC AUTONOMOUS INSTITUTION)

Accredited by NAAC with 'A+' Grade & NBA, Approved by AICTE and Affiliated to JNTUH

Cheeryal (V), Keesara (M), Medchal (Dist), Telangana – 501 301.

October – 2026 (2023-2027)

Geethanjali College of Engineering and Technology

(UGC Autonomous)

(Affiliated to JNTUH, Approved by AICTE, New Delhi)

Cheeryal (V), Keesara(M), Medchal Dist.-501 301.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CYBER-SECURITY)



CERTIFICATE

This is to certify that the Project-Based Learning (PBL) report entitled **"SECURE CLOUD FILE STORAGE USING ENCRYPTION"** has been carried out by **D.Pavani Reddy (23R11A62A4) ,D.Alekhya Ashwini (23RR1A62A6), J.Lasya Priya (23R11A62B6)**, of IV Year I Semester, B. Tech in CSE-CYBER SECURITY, during the academic year 2023-2027, under the guidance of **V. Soujenya,** Assistant Professor, Department of Computer Science and Engineering (Cyber Security). This work is submitted in partial fulfilment of the requirements for the Project-Based Learning (PBL) component of Cloud Computing.

Project Guide	**Head of the Department**
V. Soujenya,	**Krishna Jyothi**
Assistant Professor	**Associate Professor**

ACKNOWLEDGEMENT

We feel honoured and privileged to place our warm salutation to our college **"Geethanjali College of Engineering and Technology (Autonomous Institution – UGC Govt. of India)"** and our Principal **Dr. K Sagar**, Professor, who gave us the opportunity to carry out this Project-Based Learning (PBL) during our IV Year B.Tech I Semester and hone our technical skills.

We express our heartfelt thanks to our Director **Dr. Udaya Kumar Susarla**, Professor, for encouraging us in every aspect of this project and helping us realise our full potential.

We are also thankful to our Head of the Department **Dr. K. Krishna Jyothi**, Professor, for providing training and guidance, excellent infrastructure, and a conducive atmosphere for completing this PBL successfully.

We would like to express our sincere gratitude and indebtedness to our project supervisor and mentor **V. Soujenya**, Assistant Professor, for valuable suggestions and interest throughout the course of this project-based learning on blockchain security and its related domain.

We would like to thank all the supporting staff of the Department of CSE (Cyber Security), Geethanjali College of Engineering and Technology (Autonomous Campus), and all other departments who have been helpful, directly and indirectly, in making our project a success.

Finally, we would like to take this opportunity to thank our families for their support and blessings, which gave us the strength to complete this project.

D.Pavani Reddy 23R11A62A4
D.Alekhya Ashwini 23R11A62A6
J.Lasya Priya 23R11A62B6

Geethanjali College of Engineering & Technology

(UGC Autonomous)

(Affiliated to JNTUH Approved by AICTE, New Delhi)

Cheeryal (V), Keesara(M), Medchal Dist.-501 301.

DEPARTMENT OF CSE-CYBER SECURITY



DECLARATION BY THE CANDIDATES

We hereby declare that the Project-Based Learning (PBL) report entitled "**Secure Cloud File Storage using Encryption**" submitted to the Department of CSE-Cyber Security, Geethanjali College of Engineering and Technology, is a record of original work carried out by us during the academic year 2026-2027, under the guidance of **V. Soujenya**, Professor of Department of CSE-Cyber Security . We further declare that this PBL report has not been submitted to any other institution or university for the award of any degree

Name(s) & Signature(s) of the Students:

1. D.Pavani Reddy (23R11A62A4)
2. D,Alekhya Ashwini (23R11A62A6)
3. J.Lasya Priya (23R11A62B6)

PROGRAM OUTCOMES (POs) :

1. **Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialisation as specified in WK1 to WK4 to the solution of complex engineering problems.
2. **Problem Analysis:** Identify, formulate, review research literature and analyse complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
3. **Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
4. **Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8)
5. **Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognising their limitations to solve complex engineering problems. (WK2 and WK6)
6. **The Engineer and The World:** Analyse and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7)
7. **Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
8. **Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9. **Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.
10. **Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PROGRAM SPECIFIC OUTCOMES (PSOs).

PSO1: Demonstrate competency in programming and problem-solving skills and apply these skills in solving computing problems.

PSO2: Select appropriate programming languages, data structures and algorithms in combination with modern technologies and apply them in developing innovative solutions.

PSO3: Apply cryptographic algorithms for ensuring cyber security as per cyber laws, demonstrate awareness of all the security related issues and employ state of the art technologies to protect the digital assets of an organisation.

Note : Keep A tick (✓) Mark wherever there is a Mapping with Project

Title of the Project	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
CLOUD COMPUTING CHALLENGES	✓	✓	✓	✓	✓	✓		✓	✓		✓	✓	✓	✓

SUSTAINABLE DEVELOPMENT GOALS (SDG) :

1 No Poverty
2 No Hunger
3 Good Health
4 Quality Education
5 Gender Equality
6 Clean Water
7 Affordable and Clean Energy
8 Decent Work and Economic Growth
9 Industry, Innovation and Infrastructure
10 Reduce Inequalities
11 Sustainable Cities and Communities
12 Responsible Consumption and Production
13 Climate Action
14 Life Below Water
15 Life on Land
16 Peace, Justice and Strong Institutions
17 Partnerships for the Goals

Title of the Project	SDG 1	SDG 2	SDG 3	SDG 4	SDG 5	SDG 6	SDG 7	SDG 8	SDG 9	SDG 10	SDG 11	SDG 12	SDG 13	SDG 14	SDG 15	SDG 16	SDG 17
CLOUD COMPUTING SECURITY TECHNOLOGIES				✓				✓	✓	✓	✓					✓	

Geethanjali College of Engineering and Technology (Autonomous)

Accredited by NAAC with A' Grade; B.Tech: CSE, EEE, ECE accredited by NBA

Cheeryal (V), Keesara (M), Medchal District, Telangana – 501301

Team Number: B6

D.Pavani Reddy	**23R11A62A4**
D.Alekhya Ashwini	**23R11A62A6**
J.Lasya Priya	**23R11A62B6**

Abstract:

Cloud computing enables users to store, access, and manage files over the internet. However, storing sensitive information in the cloud introduces security risks, including unauthorized access, data breaches, and data tampering. This project, Secure Cloud File Storage Using Encryption, aims to address these challenges by developing a secure file-storage system that uses cryptographic techniques to protect user data. The proposed system uses AES-256-GCM encryption to encrypt files before storing them, ensuring data confidentiality and integrity. It also incorporates user authentication, access control, and secure file upload and download mechanisms to prevent unauthorized access. Additionally, integrity verification helps detect unauthorized modifications to stored files. The system can be implemented using Python, Flask, and a database, with local storage or a cloud-storage service as the storage backend. The project demonstrates how encryption and security mechanisms can improve the protection of digital assets while maintaining convenient file management. The objective is to provide a practical, cost-effective, and secure solution for cloud-based file storage.

TABLE OF CONTENTS

S.No.	Contents	Page No.
	Abstract	**1**
1	**Introduction**	**2**
1.1	**About the Project**	**2**
1.2	**Project outcomes and objectives**	**2**
2	**System Design**	**3**
3	**Flowchart**	**4**
3.1	**System Architecture**	**5**
3.2	**Modules**	**6**
4	**Implementation**	**7**
4.2	**Sample Code**	**8**
5	**Sample output Screenshots/Results**	**11**
6	**Conclusion**	**12**
7	**References**	**13**

ABSTRACT

Cloud computing can provide users with the ability to easily store, access and manage their data. However, this can cause security issues as vulnerabilities can occur when storing data in the cloud. Secure Cloud File Storage Using Encryption will provide an application that allows users to securely store their files.

This application will use AES-256-GCM encryption to provide a secure way for users to store their data. Registration, login and uploading of files are all done securely. Files can also be accessed and managed securely. The application includes authentication, authorisation and key management to reduce the chances of unauthorised access. This application has been programmed using Python for the backend with Flask. The frontend has been coded with HTML, CSS and JavaScript. The application can store user and file data using SQLite. Files can be stored in the local storage or using a cloud service. The application also includes extra features such as secure file handling, validation and activity logging.

The purpose of this project is to provide a cost-effective, practical and reliable solution for securely storing files. The application uses encryption, authentication and validation to show how cryptography can improve the security of cloud storage. Testing was carried out to ensure the application does what it should do. This includes testing the application, encryption/decryption and validation. Testing was also carried out to ensure there are no vulnerabilities to unauthorised access.

1. INTRODUCTION

Cloud computing is used to store, retrieve and manage digital data. Data can be stored in remote servers and retrieved when needed. By using cloud based storage systems there will be a more scalable, accessible and flexible solution to storing data, while reducing infrastructure costs. Storing sensitive data in the cloud can cause problems as data can be accessed without permission, stolen, leaked and tampered with.
Data can be encrypted to secure it. This uses a cryptographic algorithm and key to encrypt plain text data into cipher text. Without the key the data can't be viewed. Advanced encryption standard is a commonly used algorithm to secure digital data. AES-256-GCM can be used to encrypt and authenticate filesEven with security measures provided by the cloud service there are still risks to storing data. These risks include weak passwords, hacking, insufficient permission, poor key management and vulnerabilities. Encryption can't prevent all attacks. A secure storage application will require strong security measures, including user authentication, user authorisation, file handling, secure key storage and integrity.The aim of this project is to create a secure application using cryptographic methods to secure users files. The application includes sign up, log in, upload, download and file management. Data is encrypted before being stored. Authorised users will have their files decrypted. The application can be stored locally or integrated with a cloud storage service.

The project will focus on the security features that are important. This will show how encryption, authentication and integrity can be used to reduce the risk of files being compromised. The application will be tested using functional and security testing.This report will explain how the application was designed, implemented, how security features were included, how the application was tested and how the project has limitations. It will show how encryption and security can be implemented to keep files secure. Key management will be discussed.

2.1 ABOUT THE TOPIC

Multiple security measures should be used to keep files secure in the cloud.Confidentiality stops unauthorised users from being able to view the data. This can be done using encryption such as AES-256-GCM.Integrity can be used to detect if a user has made changes to the data. This can be done using AES-GCM to authenticate the data using an authentication tag. This means the application can detect certain changes made to cipher text.Authentication can be used to make sure users are who they say they are. This can be done by requiring the user to sign up to an account. Passwords can be stored as hashes and users can be verified.Authorisation will determine which files users can view, download and delete. Having proper authorisation will stop users from viewing files from other accounts.Key management is important. Even if the data is encrypted with strong encryption if the keys are exposed it won't be secure. A secure

key management is required.Files should be securely managed. This can be done by including validation, limiting file size, validating file names and handling errors.

This project will show how security can be implemented into an application to keep files secure. Risks such as unauthorised users accessing the files, users accounts being hacked, files being tampered with and keys not being managed securely will be considered. This project will show how encryption, authentication, authorisation and integrity can be used to reduce the risks.The application will be created using Python and Flask for the back end. HTML, CSS and JavaScript will be used for the front end. User and file data will be stored using SQLite. The AES-256-GCM can be implemented by using the cryptography library in Python. The files can be stored locally or using a cloud storage service.
The project will be implemented and tested using functional and security testing. Practical factors will be considered such as cost, storage size, performance, usability and security limitations.

2.2 PROJECT OUTCOMES AND OBJECTIVES

Main objectives of this project are:
Secure File Storage – Create an application that will store encrypted files to keep sensitive information secure.
Encryption – Implement AES-256-GCM to encrypt and decrypt files.
Authentication – Implement a sign up and log in system to verify users.
Authorisation – Ensure users can only view the files they are authorised to view.
File Management – Implement file upload and download with proper validation.
Integrity – Verify that the encrypted data has integrity by ensuring the data can be authenticated when decrypting. Check that tampering can be detected by ensuring the authentication tag isn't valid.
Key Management – Implement a secure method for key generation, storage and access to reduce the risk of keys being exposed.
Security Testing – Test the application using encryption, decryption, unauthorised access, file integrity, invalid data and file management.
Cost Effective – Create and test the application using free and open source software. Files can be stored locally so there is no cost to developing the application.

OUTCOMES

This project shows how encryption and access control can be used to increase security in a file management application. Multiple security measures will be used.The files will be encrypted to keep the data secure. Users will be authenticated and authorised to access secure sections of the application. .

2. SYSTEM DESIGN

The project will use a layered security model to identify and address any security issues that can occur with a cloud-based file storage project. The model will use 5 layers: User Interface layer, Application layer, Authentication & Access control layer, Encryption layer and Storage layer.



Fig 1: Secure encryption based cloud file storage using a layered architecture.

1. User Interface Layer: This layer provides functionality for users to interact with the application. Users can register, login, upload, download and manage files. Threats could be improper usage of sessions, cross-site scripting (XSS) and malicious inputs.

2. Application Layer: This layer handles requests from users and performs file operations. Possible security flaws could be poor file management, improper error handling and unauthorised API access

3. Authentication & Access Control layer: This layer validates the user and checks what files they can access. Threats could be brute force attacks on login, account compromise and insecure access control.

4. Encryption layer: This layer provides encryption for the files. When authorised users try to access the files they will be decrypted. Threats could be key exposure, nonce repetition and incorrect encryption. These threats can be protected by using AES-256-GCM encryption, using unique nonces on each encryption using the same key, secure key management and validating the authentication tag.

5. Storage layer: This layer stores the encrypted files along with its meta data locally or to a cloud storage service. Threats could be unauthorised access to files, deletion of files, tampering with data and incorrect configuration of storage.

3. FLOWCHART

This is a systematic way of developing and testing a secure cloud file storage application, including identifying what needs to be secured, analysing possible threats and designing, implementing and testing your system.It shows a clear process for identifying vulnerabilities within your application, ensuring you have successfully implemented the security features and gives you a clear view of how the security requirements have been implemented and tested.

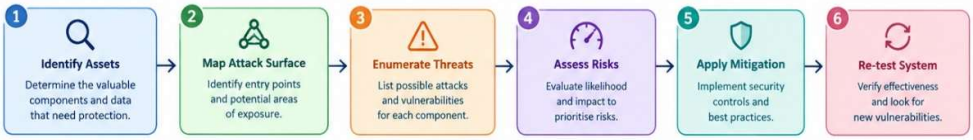


Fig 2 – The development and testing of a secure cloud file storage application

The flow chart should include:

- **Identify Assets** – Identify what files, user accounts, encryption keys and metadata needs to be secured.
- **Analyse Security Threats** – Identify any potential threats to the security of the application (including unauthorised access, data leakage, file tampering and encryption key exposure).
- **Design** – Define how the application will be structured, what the database will look like, how authentication, encryption and file storage will be handled.
- **Implement Security** – Implement authentication for users, access control, AES-256-GCM encryption and decryption, and secure file storage.
- **Encrypt & Decrypt Files** – Encrypt the files and store them. When a user tries to download the file it will be decrypted.
- **Test Security** – Test that encryption / decryption is working, files are being verified, access controls are working, invalid inputs are being handled and unauthorised access is being prevented.
- **Analyse & Improve** – Analyse the results of your tests, fix issues identified and re-test.

4.1 SYSTEM ARCHITECTURE MAPPING

This will identify important elements of the secure file storage application including what assets need to be secured.

- **Assets:** The main assets to be secured are the files uploaded by users, encryption keys, user details, file metadata, authentication details and logging. All of these should be protected against unauthorised access, modification, disclosure and loss.

- **Security Assumptions:** It is assumed that users will keep their login details safe, encryption keys will be kept secure and that authentication, authorisation, encryption and decryption will be properly implemented. If these assumptions are not kept then the security of the application could be compromised.
- **Authentication:**This is used to confirm a user is who they say they are. Passwords should be stored using a suitable password hashing algorithm.
- **Authorisation:**Ensures the user can only access and manage the files they are authorised to. For each secure function, the file should be checked to make sure it is owned by them and they have the correct permissions.
- **Encryption & Decryption:**This will use AES-256-GCM to encrypt the files. The encryption must be implemented using a unique nonce for each encryption using the same key. The decryption process must check the authentication tag.
- **Encryption Key:**This will be used to generate, store and implement the encryption keys. The keys should be kept secure and shouldn't be stored with the encrypted files.
- **File Storage:**This will be used to store the encrypted files along with the metadata. During development the application may store the files locally, however it can be integrated with a cloud storage service.
- **File Verification:**This is used to help identify if the encrypted file has been modified by an unauthorised user. During decryption the authentication tag from AES-GCM will be checked. If the file fails the authentication then it should be rejected.
- **Logging:**This will be used to log important events such as successful logins, uploading, downloading and deleting of files. It shouldn't display any passwords, encryption keys or sensitive information from the files.
- **Data Flow:**When a user uploads a file the application will authenticate them, validate the file, encrypt the contents and store the ciphertext. When a user tries to download the file the application will check they are authorised to download the file, retrieve the encrypted file then check the authenticity of the file during decryption. The original file will be provided to the authorised user.

4.IMPLEMENTATION

This section outlines the security features that will be implemented to protect user files. This includes authentication, access control, encryption and decryption, secure storage and integrity verification. Examples of code will also be provided to show how files can be encrypted and decrypted using AES-256-GCM in Python. Examples of security testing have been included along with a table showing the main security issues.The application can be built using Python for the backend with Flask and HTML, CSS and JavaScript for the frontend. Using SQLite will allow us to store user and file information. cryptography can be used to encrypt the file using AES-256-GCM. Files can be stored locally however there can also be integration for cloud storage.
These security features will be implemented to ensure that files are stored securely. Security analysis and testing will be done to ensure that everything has been implemented correctly. All results in the final project will be based on the testing completed.

5.1 Module Analysis

5.1.1 User Authentication Module

This module will be used to confirm the identity of any users wishing to access the application. This will ensure that only authorise users can access the secured features by registering and logging in.
User information will be collected during registration. The password will be hashed using the correct password hashing method. Passwords must not be stored in clear text or using any reversible encryption. When a user tries to log in the password will be checked against the stored hash.
After confirming the user, a secure session will be created. This will allow the user to access the authorised features. Sessions must be handled correctly to avoid theft.
Secure password hashing, input validation, secure sessions, limiting login attempts and error handling will be used to secure this module. Although this is the first line of defence the users must also be authorised.

5.1.2 File Encryption Module

This module will be one of the most important parts of the system. This will ensure that uploaded files are encrypted. This will turn the file content into ciphertext.
Files will be encrypted using the Advanced Encryption Standard (AES) with a 256-bit key and using Galois/Counter Mode (GCM). AES-256-GCM is an authenticated encryption algorithm so we can ensure the files are confidential and verify that the data is authentic.
The file will be processed as follows:

- Valid file is received from authenticated user.
- Read file content.
- Get secure encryption key.
- Create unique nonce.
- File is encrypted using AES-256-GCM. This will produce ciphertext and authentication tag.
- Store nonce with ciphertext in a suitable format.
- Pass encrypted file to storage module.

Every encryption must use a unique nonce. If the same nonce is used with the same AES-GCM key then this can cause major security issues.
The encryption key must be secured. They must not be stored with the encrypted files. To make this more advanced we can use envelope encryption. This will generate a unique data encryption key for every file which will be secured with a key encryption key.

5.1.3 File Download & Decryption Module

This module will allow authorised users to download their files.
When a user requests to download a file the application will first confirm that they are authenticated. Then it will confirm that they own the file or have the correct access.
Once they have been authorised the file will be retrieved. This will then be passed to the decryption module along with the encryption key. The file will then be authenticated and decrypted using AES-256-GCM.

The file will be processed as follows:

- User requests to download a file.
- Application will confirm that they are authenticated and authorised.
- Retrieve the encrypted file.
- Get correct decryption key.
- Authenticate and decrypt using AES-GCM.

If the file has been authenticated the original file will be available for download.If the file hasn't been authenticated then the corrupt/modified data will be rejected.This will help prevent users downloading files they shouldn't have access to. It will also tell us if the encrypted data has been altered.No sensitive information should be shown in error messages. We should also ensure that files aren't left in public temporary folders after they have been decrypted.

5.1.7 File Integrity Module

This module will help us to identify if the encrypted data has been altered. AES-256-GCM provides authenticated encryption. During decryption the cryptographic library will check the authentication tag. This should fail if the ciphertext, nonce or authentication tag have been altered. The application will then reject the file.This is especially important if the encrypted files are being stored remotely.If the authentication fails then this should be seen as a security issue. Partially decrypted data shouldn't be returned.We can use a cryptographic hash if we need to compare different versions of files. Storing the hash alongside an unsecured file will not prevent an attacker from replacing the file and hash.

5.1.8 Key Management Module

This module will contain key generation, securing, retrieval and usage.It is very important to secure the keys to ensure the encrypted storage is safe. An attacker could potentially decrypt the original file content if they obtain the decryption key.Keys should be generated using a cryptographically secure random number generator. AES-256 requires a 256-bit key (32 Bytes).Keys should be stored separately from the encrypted content. They should be secured using the correct access controls. In a production environment we may want to use a dedicated key management service or hardware backed key encryption.
We should not expose the keys in logs, errors, source code or public configuration files. We may also want to consider how we back up and recover the keys.

5.2 Code Examples

Below are two examples of how to implement encryption/decryption using AES-256-GCM using the cryptography library in Python.
Example 1: Encrypt File

```
import os
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
def encrypt_file(input_path, output_path, key):
# Create unique nonce
nonce = os.urandom(12)
# Open file
with open(input_path, "rb") as file:
plaintext = file.read()
# Encrypt file
aesgcm = AESGCM(key)
ciphertext = aesgcm.encrypt(nonce, plaintext, None)
# Save nonce, ciphertext and tag
with open(output_path, "wb") as file:
file.write(nonce + ciphertext)
# Create 256 bit key
key = AESGCM.generate_key(bit_length=256)
```

```
encrypt_file("sample.txt", "sample.txt.enc", key)
```
The code creates a unique nonce of 12-bytes. It then reads the contents of the file. The file is then encrypted using AES-256-GCM.
The ciphertext will include the authentication tag. The nonce is stored at the beginning of the file so it can be retrieved during decryption.

Example 2: Decrypt File

```
from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
def decrypt_file(input_path, output_path, key):
# Open encrypted file
with open(input_path, "rb") as file:
encrypted_data = file.read()
# Get first 12-bytes of data (nonce)
if len(encrypted_data) < 28:
raise ValueError("Invalid encrypted file.")
nonce = encrypted_data[:12]
ciphertext = encrypted_data[12:]
# Authenticate and decrypt
aesgcm = AESGCM(key)
try:
plaintext = aesgcm.decrypt(
nonce,
ciphertext,
None
)
except InvalidTag:
raise ValueError(
"Failed to authenticate file. File may have been tampered with or incorrect key used."
) from None
# Save plaintext
with open(output_path, "wb") as file:
file.write(plaintext)
decrypt_file("sample.txt.enc", "recovered.txt", key)
```

5.3 Security Testing

Security testing must be done to ensure that the application is performing the required functions and implementing the security requirements.Functional and security testing can be performed on the system. All testing should include what was inputted, expected outcome, actual outcome and whether it passed.
Encrypt & Decrypt File: Upload a sample file. Encrypt and decrypt using correct key. Ensure that the recovered file matches the original.
Expected Outcome: Decrypted file should match the original file.
Unauthorised Access: Try to access secured features without logging in.
Expected Outcome: Application should reject the request.
File Ownership: Create 2 user accounts. Upload different files. Log into one account and try to access the other files.
Expected Outcome: Application should not allow access to files they don't have permission to.
Tamper With File: Alter a copy of the encrypted file. Try to decrypt.
Expected Outcome: File should fail authentication using AES-GCM. Application will reject the data.
Incorrect Key: Try to decrypt encrypted file using incorrect key.

Table 1: Summary of Security Issues With Secure Encrypted Cloud File Storage.

Security challenge	Underlying mechanism or weakness	Mitigation measures
Unauthorized access	Weak authentication or missing authorization checks	Secure password hashing, session protection, and server-side access control
Data disclosure	Files stored or transmitted without appropriate protection	AES-256-GCM encryption and secure communication
File tampering	Modification of stored or transmitted encrypted data	Authenticated encryption and authentication-tag verification
Encryption-key exposure	Keys stored insecurely or exposed through application components	Secure key storage, restricted access, and appropriate key-management practices
Password compromise	Weak passwords or unsafe credential storage	Password hashing, login-attempt controls, and secure session management
Insecure file uploads	Unsafe filenames, excessive file sizes, or insufficient validation	Server-side validation, size limits, and safe storage identifiers
Broken file access control	Failure to verify file ownership or permissions	Server-side authorization checks for every protected operation
Storage misconfiguration	Excessive permissions or public exposure of stored files	Private storage permissions and restricted access credentials
Data loss	Accidental deletion, storage failure, or corrupted files	Appropriate backups, recovery procedures, and storage-error handling
Inadequate monitoring	Missing records of important application events	Security-focused activity logging and regular review
Improper error handling	Sensitive details exposed through error messages or failed operations	Generic external error messages and secure internal logging
Excessive resource consumption	Large uploads or repeated requests exhausting available resources	Upload-size limits, request restrictions, and appropriate resource management

The table shows that secure file storage requires multiple complementary mechanisms. Encryption protects file contents, authentication verifies user identity, authorization limits access, integrity verification detects certain forms of tampering, and key management protects the ability to decrypt stored data.

5.5 Implementation Summary

This project combines cryptographic techniques and application-level security mechanisms to develop a secure file-storage solution. The proposed design uses AES-256-GCM to protect file contents, authentication to verify users, access control to restrict file operations, and integrity verification to detect modifications to encrypted data.

The implementation also highlights the importance of encryption-key management, secure file handling, and appropriate storage permissions. These elements must work together to reduce the risks associated with storing sensitive information in local or cloud-based environments.

The project's effectiveness should be evaluated through the functional and security tests described in this section. Actual performance, test results, and implementation limitations should be documented based on the application that is ultimately developed.

6. SAMPLE OUTPUTS SCREENSHOTS

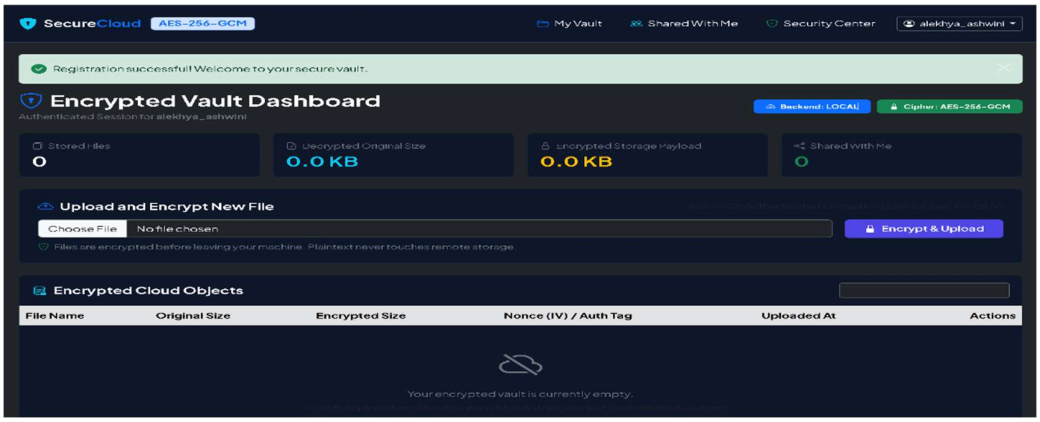


FIG 6.1 HOME SCREEN

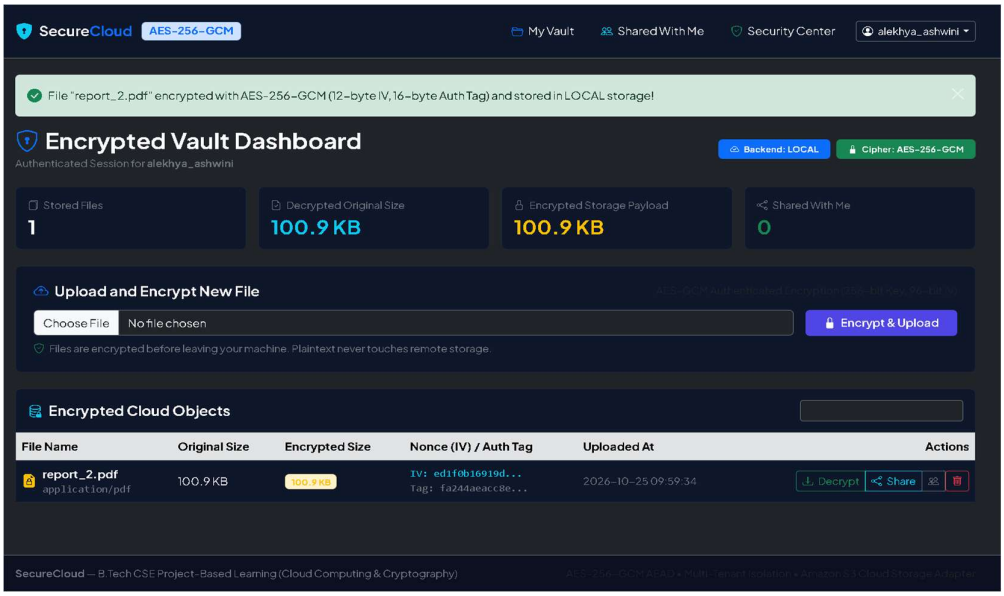


FIG 6.2 FILE SELECTION

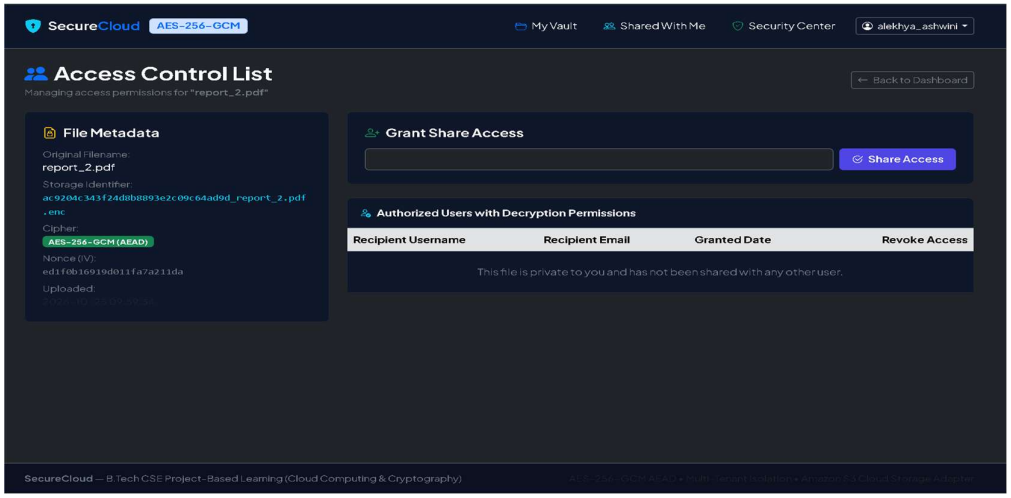


FIG 6.3 FINAL OUTPUT

7.CONCLUSION

Cloud computing offers a convenient, scalable service for storing, accessing and managing digital data. However, due to the nature of cloud computing, storing sensitive files within a cloud environment poses many potential security issues, including unauthorised access, data leaks, data manipulation and key leaks. Therefore, additional security mechanisms such as cryptography, authentication, authorisation and appropriate key management should be implemented.

This project, Secure Cloud File Storage Using Encryption, aims to provide solutions using file encryption and various security mechanisms. This system will implement AES-256-GCM to securely store the data, ensuring the authenticity of the decrypted data. In addition, authentication and authorisation will be applied to secure the application. As well as secure file handling, appropriate key management will be applied.

Potential security risks identified within this project were weak authentication, unauthorised file access, insecure key storage, data manipulation and incorrect storage permissions. The system includes or suggests secure password hashing, authorisation checks on the server-side, authenticated encryption, unique nonce creation, secure key management and data integrity. These can be tested using functional and security testing.

The system can be developed using free and open-source tools, allowing implementation and testing without a mandatory paid cloud subscription. Local storage can be used during development, while a cloud-storage backend can be integrated if remote storage is required.

8.REFERENCES

1. National Institute of Standards and Technology (NIST), *Advanced Encryption Standard (AES)*, Federal Information Processing Standards Publication 197, 2001. Available: https://doi.org/10.6028/NIST.FIPS.197-upd1

2. National Institute of Standards and Technology (NIST), *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST Special Publication 800-38D, 2007. Available: https://doi.org/10.6028/NIST.SP.800-38D

3. National Institute of Standards and Technology (NIST), *Digital Identity Guidelines: Authentication and Lifecycle Management*, NIST Special Publication 800-63B. Available: https://pages.nist.gov/800-63-4/sp800-63b.html

4. National Institute of Standards and Technology (NIST), *Recommendation for Key Management: Part 1 – General*, NIST Special Publication 800-57 Part 1, Revision 5, 2020. Available: https://doi.org/10.6028/NIST.SP.800-57pt1r5

5. OWASP Foundation, *OWASP Application Security Verification Standard (ASVS)*. Available: https://owasp.org/www-project-application-security-verification-standard/

6. OWASP Foundation, *File Upload Cheat Sheet*. Available: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

7. OWASP Foundation, *Password Storage Cheat Sheet*. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

8. Python Cryptographic Authority, *Cryptography Documentation: Authenticated Encryption*. Available: https://cryptography.io/en/latest/hazmat/primitives/aead/

9. Pallets Projects, *Flask Documentation*. Available: https://flask.palletsprojects.com/

10. Amazon Web Services, *Amazon Simple Storage Service (Amazon S3) Documentation*. Available: https://docs.aws.amazon.com/s3/